当代数据管理系统第一次项目报告

轻问卷：基于 MongoDB 的在线问卷系统

GitHub 仓库：https://github.com/DawnECNU/---518

姓名	余冬雨
学号	10255501518
完成日期	2026 年 10 月 22 日

项目采用 Flask、PyMongo 与 MongoDB 实现，包含账号、问卷管理、条件跳转、填写校验和统计功能。

1 项目目标

本项目的目标是完成一个可以直接运行的在线问卷系统。登录用户可以创建、编辑、发布和关闭问卷，并通过公开链接收集答案；填写者登录后可以逐题作答，系统根据答案执行条件跳转，并在提交前完成必要校验。

项目主要目标如下：

- **业务目标：**支持问卷创建、发布、填写、关闭和统计的完整流程。
- **题型目标：**支持单选、多选、文本填空和数字填空四类题目。
- **流程目标：**支持必答、选答、条件跳转、断点续填和最终提交。
- **数据目标：**使用 MongoDB 保存用户、问卷、题目和答卷，并建立必要约束与索引。
- **质量目标：**通过自动化测试和真实 HTTP 流程验证主要功能。

课程学习的重点是将 MongoDB 真正用于业务系统，而不只是完成页面。因此项目还将题目结构、选项、跳转规则和答卷路径作为建模对象，并分别讨论嵌入、引用、校验和索引的使用。

完成范围

模块	完成内容
账号	注册、登录、退出、密码哈希、用户数据隔离
问卷	创建、编辑、发布、关闭、公开链接、截止时间
题目	单选、多选、文本、数字及题型专属限制
填写	逐题保存、条件跳转、检查页、最终提交
统计	选项分布、回答人数、文本答案、数字均值与范围
测试	业务单元测试、集成测试、CSRF 测试和 HTTP 冒烟测试

表 1 项目完成范围

2 系统设计

2.1 总体结构

系统采用 Flask 服务端模板，浏览器提交 HTML 表单，Flask 路由负责登录状态、权限和页面跳转，survey_logic.py 负责题目、答案、跳转和统计等业务规则，PyMongo 负责访问 MongoDB。页面、路由和领域逻辑分开后，关键规则可以脱离Web 环境进行单元测试。

浏览器 -> Flask 路由 -> 业务逻辑 -> PyMongo -> MongoDB

2.2 模块划分

文件	职责
app.py	应用入口、账号、问卷、填写、统计和请求校验
survey_logic.py	题目构造、答案校验、跳转匹配和统计计算
schema.py	四个集合的 JSON Schema
db.py	数据库连接、校验器和索引初始化
templates/	注册、登录、问卷、填写、检查、统计等页面
tests/	单元测试、集成测试和真实 HTTP 冒烟脚本

表 2 主要模块

2.3 主要流程

用户创建草稿问卷后添加题目和跳转规则，系统检查题目及规则是否合法，满足条件后发布。填写者登录并创建答卷，系统保存当前题号和答案。每提交一题，先执行答案校验，再匹配跳转规则；到达结尾后进入检查页，最终提交时再次校验实际访问过的题目。

问卷状态分为草稿、已发布和已关闭；答卷状态分为填写中和已提交。问卷发布后，题目、选项和跳转规则被锁定，避免已有答卷与题目结构不一致。超时或已关闭的问卷不再接受新的填写进度。

3 MongoDB 设计

3.1 建模思路

系统使用 users、surveys、questions 和 responses 四个集合。用户、问卷、题目和答卷之间通过 ObjectId 引用；题目选项和跳转规则属于题目内部的强相关数据，直接嵌入 questions 文档；答卷答案以题目 ID 为键保存在 answers 对象中。

这种设计避免把所有内容放入一个大型问卷文档，也没有把每个选项拆成独立集合。问卷和题目可以按业务对象整体读取，答卷可以单独增长、筛选和统计。

3.2 集合与字段

集合	主要字段	说明
users	username、password_hash、created_at	保存账号信息，密码只保存加盐哈希
surveys	owner_id、title、status、public_id、deadline	保存问卷设置、发布状态和公开链接标识
questions	survey_id、order、type、options、constraints、jump_rules	题目顺序、选项、限制和跳转规则
responses	survey_id、user_id、answers、path、current_question_id	答卷内容、实际路径、当前进度和状态

表 3 核心集合

3.3 校验与索引

应用启动时为四个集合安装 JSON Schema，校验级别为 strict，校验动作为 error。Schema 限制必填字段、数据类型、状态枚举、长度和题号范围。业务代码负责给出友好错误，数据库负责阻止不合规文档写入。

主要索引包括：username 唯一索引；public_id 部分唯一索引；(survey_id, order) 唯一索引，保证同一问卷题号不重复；responses 上的 survey_id/status 和 survey_id/user_id/status 复合索引，用于统计、恢复答卷和重复填写限制。

3.4 一致性与匿名

更新答卷时，系统将 current_question_id 同时放入查询条件，避免双击或双窗口导致答案错位。匿名填写仍要求登录，文档内部保留 user_id 用于访问控制和重复填写判断，但统计页面不展示填写者身份。

4 跳转设计

跳转规则保存在每道题的 jump_rules 数组中，每条规则包含条件和目标。系统按数组顺序匹配，第一条命中的规则生效；没有命中时进入题目顺序中的下一题。

题型或条件	跳转方式
单选	选择指定选项时跳转
多选	包含指定选项时跳转
多选	选择数量等于、至少或至多指定值时跳转
数字	数值位于指定区间时跳转
任意题目	target 为 null 时直接结束问卷

表 4 跳转条件

保存规则和发布问卷时都会检查目标题目是否存在，并要求目标题目的 order 大于当前题目。只允许向后跳转可以从结构上避免循环。发布后题目结构锁定，避免规则引用的选项被删除或修改。

答卷的 path 只记录实际访问过的题目。被跳过的题目不进入 path，因此不会被当作必答题缺失。例如有 Q1 到 Q4 四道题，Q1 选择 A 时跳到 Q4，则 path 为[Q1, Q4]；选择 B 且没有命中规则时，默认进入 Q2。

5 校验设计

系统采用多层校验。页面层的 required、minlength、maxlength 和 number 属性用于即时提示；路由层检查登录、权限、问卷状态和截止时间；业务层检查题型和答案规则；MongoDB 的 JSON Schema 和索引作为最后的数据约束。

校验对象	主要规则
账号	用户名 3-32 字符、不能有空格；密码 6-128 字符；两次密码一致
问卷	标题非空且不超过 200 字符；截止时间必须晚于当前时间
题目	标题非空；选择题至少 2 个、最多 20 个选项且不重复
单选	答案必须是已有选项 key
多选	选项有效且不重复；检查最少、最多或固定数量
文本	检查最少和最多字符数
数字	检查有限数值、整数限制、最小值和最大值
提交	按 path 重新检查所有实际访问过的题目

表 5 校验规则

所有 POST 请求默认检查 CSRF Token，并使用常量时间比较。会话 Cookie 设置为 HttpOnly 和 SameSite=Lax。问卷管理、题目编辑和统计接口均检查 owner_id，用户不能访问或修改他人的问卷。

6 AI 使用过程

本项目使用 AI 辅助需求梳理、代码检查、错误分析和测试补充。需求取舍、代码运行、结果确认和最终提交由本人完成。AI 主要用于提供排查思路和局部实现建议，所有结论都回到本地代码和测试结果中核对。

6.1 实际案例：创建问卷时报错

项目早期出现过点击创建问卷后返回服务器错误、无法进入问卷详情页的问题。一开始只修改页面，没有解决。后来在 AI 辅助下，将问题拆成页面输入、Flask 路由、PyMongo 连接和 MongoDB Schema 四层逐一检查。

排查过程如下：

- 查看 Flask 终端堆栈，确认异常发生在创建问卷的请求处理阶段。
- 查看 MongoDB 返回信息，区分连接问题、唯一索引冲突和文档校验失败。
- 对照 surveys 的必填字段、可空字段和 bsonType，检查初始化文档是否一致。
- 确认 MONGO_URI 和 MONGO_DB_NAME 指向正确实例，并使用最小数据测试插入。

最终发现新建问卷的初始化文档与数据库校验规则或连接配置不一致。修复时补齐初始化字段，统一日期和可空字段的类型；应用启动时重新安装匹配的 Schema 和索引；确认连接配置后重启。随后先测试创建单份问卷，再运行完整测试和真实HTTP 冒烟流程。

6.2 使用说明

AI 适合帮助整理问题、提供检查清单和生成测试场景，但不能替代实际运行。使用过程中不提交真实密码、问卷答案或敏感连接信息；AI 给出的版本号、函数行为和测试结论都经过本地复核。能够独立解释关键逻辑并完成测试，才认为相关代码可以保留。

7 测试结果

测试环境为 Python 3.12.8、MongoDB 8.3、Flask 3.1.3、PyMongo 4.18.1。测试分为业务单元测试、Flask 集成测试和真实 HTTP 冒烟测试。

测试内容	结果
账号注册、登录和问卷隔离	通过
创建问卷、添加四类题目、发布问卷	通过
单选、多选、数字区间条件跳转	通过
必答、文本长度、多选数量、数字范围与整数校验	通过
逐题保存、检查页和最终提交	通过
整卷统计与单题统计	通过
匿名、重复填写、关闭问卷和截止时间	通过
CSRF 防护	通过

表 6 测试内容

最终执行 python -m unittest discover -s tests -v，共运行 10 个测试方法，覆盖 21 个检查点，结果为 Ran 10 tests in 2.845s，OK。

此外，使用真实 HTTP 会话执行了注册、创建问卷、添加题目、发布、开始填写、提交答案、提交问卷和查看统计的完整冒烟流程，结果为成功。

8 遇到的问题与解决

问题	解决方法
创建问卷时报错	分层查看 Flask 和 MongoDB 错误，统一初始化字段与 Schema，确认连接后回归测试
跳转可能形成循环	保存和发布时要求目标 order 大于当前题，target=null 表示结束
跳过的必答题被误判	使用 path 记录实际经过的题目，提交时只校验 path 中的题目
编辑选项破坏已有规则	尽量复用选项 key，删除被规则引用的选项时拒绝保存
并发更新答卷错位	更新条件同时包含 current_question_id，冲突时回到最新当前题
Windows 找不到时区	加入 tzdata，日期统一按 UTC 保存，显示时转换为 Asia/Shanghai
匿名与访问控制冲突	文档内部保留 user_id，统计层不展示身份，匿名标志固化在答卷中

表 7 主要问题与解决

这些问题说明，问卷系统不能只依赖页面校验。关键规则需要在业务逻辑中执行，数据结构需要由数据库再次约束，异常流程需要用测试固定下来。

9 总结

本项目完成了在线问卷系统的主要闭环：用户创建和发布问卷，填写者按规则作答，系统保存进度并完成校验，创建者查看统计。MongoDB 的文档模型适合保存题目选项和答卷答案，JSON Schema 与索引为数据一致性提供了额外保障。

当前系统仍有可扩展空间，例如增加更多题型、使用聚合管道优化统计、增加数据导出、补充高并发测试和移动端测试。作为阶段一项目，现有实现和测试结果已经能够支撑后续迭代。